Packaging Errors in the SpoilFree Chrome Extension: Missing Icon and Stylesheet Assets

Divyam Saluja

GitHub: salujadivyam | LinkedIn: salujadivyam

Abstract

Loading the SpoilFree Chrome extension as an unpacked extension failed because the manifest referenced icon assets at paths that did not match their location in the project and it declared a blur stylesheet that was absent. This report documents the observed symptoms, likely causes supported by the project investigation, corrections made to the manifest and asset set and the verification that was completed. The icon paths were updated to the icons/ directory, styles.css was recreated with the spoiler-blur rule, the manifest version was raised to 1.2 and content-script ordering was adjusted so helper scripts load before content.js. The edited manifest was parsed as valid JSON and the developer confirmed that the extension loaded in Developer mode. A repeatable packaging checklist is also proposed to reduce the likelihood of similar failures.

Index Terms, Chrome extension, Manifest V3, packaging, asset paths, content scripts, stylesheet, debugging.

I. INTRODUCTION

Browser extensions depend on a manifest to describe the resources and scripts that make up the extension package. In a Manifest V3 extension, the manifest identifies files such as icons and content-script stylesheets. These declarations must agree with the files that are actually present in the project directory. A filename can exist somewhere in the project and still be unavailable to Chrome if the manifest points to a different path.

SpoilFree is a Chrome extension intended to blur spoilers for titles on a user's watchlist. The packaging issue discussed here arose while loading the extension as an unpacked extension. The investigation identified two asset-level mismatches: icon paths did not reflect the directory structure and the declared stylesheet was missing. The report focuses on those observed failures and the corrections recorded during debugging. It does not claim that a full Chrome Web Store release or testing across multiple Chrome profiles was completed.

Section II describes the errors. Section III explains why the asset references failed. Section IV records the changes. Section V covers verification and its limits. Section VI gives a checklist, followed by the conclusion and references.

II. OBSERVED ERRORS

A. Icon Path Error

Chrome reported: “Could not load icon ‘icon16.png’ specified in ‘icons’. Could not load manifest.” The manifest initially named icon16.png, icon48.png and icon128.png at the extension root. During investigation, the icon files were found under an icons/ subfolder. Consequently, the manifest's relative paths did not match the files' actual locations.

This is a path-resolution problem rather than evidence that the image contents were invalid. A manifest entry must identify the intended file using a path relative to the extension package. If the path points to the wrong directory, Chrome cannot retrieve the referenced asset even if the file is present elsewhere. Updating all three entries consistently was therefore necessary; changing only one icon entry would have left the remaining declarations inconsistent.

B. Missing Stylesheet

The manifest also listed styles.css as a stylesheet for the content script, but the developer reported that no styles.css file existed in the project. The stylesheet was intended to define the visual blur applied to spoiler content. This created a second mismatch between the manifest and the packaged files.

The consequences differ from those of an incorrect icon path. A missing file named by the manifest can prevent the extension from loading successfully. Even if the extension's other components were available, the intended blur presentation would not be supplied by the absent stylesheet. The missing file therefore affected both package completeness and the feature's visual behavior.

C. Scope of the Failure

Both errors concerned declared resources, not a demonstrated failure of the spoiler-detection logic itself. The available investigation supports a narrow conclusion: Chrome could not load the package while the declared asset paths and files were inconsistent. It does not establish that every feature of SpoilFree was tested after the correction.

III. TECHNICAL ANALYSIS

A. Manifest and Project Tree Must Agree

The manifest acts as a resource map for the extension. Its icon entries identify the images used by Chrome, while the content-script configuration identifies scripts and stylesheets to apply to matching pages. These declarations are not merely descriptive metadata: they refer to concrete files that must be included in the package at the specified relative paths.

A useful diagnostic is to compare each manifest path against the project tree, character by character. The comparison should include the filename, extension, folder name and relative location. Case differences may also matter when a package is used in environments with case-sensitive file

systems. This last point is a general preventive consideration; the reported incident specifically involved the root path versus the icons/ directory.

B. Stylesheet Role in the Blur Effect

The intended stylesheet defines the .spoilfree-blur class. The recorded correction applies a 6 px blur, uses a pointer cursor, disables text selection and reduces the blur to 4 px on hover. These rules are part of the presentation layer: they determine how marked content appears and how a user can interact with it. They do not, by themselves, decide which text qualifies as a spoiler.

Keeping the stylesheet in the package is therefore essential to the expected interface. A manifest entry alone cannot create the CSS rules it references. The file must exist, be named correctly and contain the expected selector and declarations.

C. Script Ordering

The correction also reordered the content-script list so helper scripts load before content.js. This reflects a dependency principle: a script that relies on functions or values established by another script should not be executed before those dependencies are available. The available notes record that ordering was changed, but do not identify every helper script or provide runtime traces; this report therefore does not infer additional dependency details.

IV. CORRECTIONS IMPLEMENTED

The following changes were recorded during the fix:

- Icon paths: changed the manifest entries to icons/icon16.png, icons/icon48.png and icons/icon128.png.
- Stylesheet restoration: recreated styles.css and added the .spoilfree-blur rule with a 6 px blur, pointer cursor, disabled text selection and a 4 px blur on hover.
- Manifest version: set the version to 1.2. The stated reason was that the store listing was at 1.1 and an update needed a higher version.
- Content-script order: arranged helper scripts before content.js.

These edits address the specific inconsistencies identified in the investigation. The icon declarations now match the reported directory structure, the declared stylesheet is present again, the manifest version is higher than the previously listed version and the recorded script ordering reflects the intended dependency sequence.

A. Illustrative Asset Audit

A small audit table can make the relationship between the declarations and files explicit. The entries below summarize the corrected paths reported for this incident; they are not a substitute for checking the actual package before each release.

Resource	Manifest / package expectation	Recorded status
16 px icon	icons/icon16.png	Path corrected
48 px icon	icons/icon48.png	Path corrected
128 px icon	icons/icon128.png	Path corrected
Blur stylesheet	styles.css	Recreated
Manifest version	1.2	Updated

Table I. Summary of the recorded packaging corrections.

V. VERIFICATION AND LIMITATIONS

The edited manifest was parsed as valid JSON in the authoring environment. Following replacement of the manifest, the developer confirmed that the extension loaded in Developer mode. These checks provide evidence that the corrected manifest was syntactically valid and that the reported load failure was resolved in the developer's environment.

The verification scope has limits. Loading in a real Chrome profile could not be reproduced in the authoring environment. The recorded evidence also does not describe a full regression suite, testing across different websites or a published Web Store release. Accordingly, the result should be stated as a successful Developer mode load reported by the developer, rather than as proof of production-wide compatibility.

For a stronger release check, the package should be loaded in a clean Chrome profile and tested on a page where the spoiler-blur behavior is expected to run. The tester should confirm that the extension loads without manifest errors, that the icons resolve, that the stylesheet is requested successfully and that the blur class changes presentation as intended. These are recommended follow-up checks, not tests claimed as completed in this incident.

The failure also illustrates why packaging checks should be treated separately from feature development. A feature may work during local coding while the packaged extension still fails before its scripts can run. In this case, the declared icon files and stylesheet were part of the load-time contract between the manifest and the project tree. Testing only the source code would not reliably detect a missing declared file.

A useful way to classify the incident is by failure stage. The icon-path mismatch and absent stylesheet are package-integrity issues because the referenced resources could not be resolved as declared. The stylesheet's blur rule is a presentation concern because it determines how spoiler text is displayed. Script ordering is a dependency concern because dependent code should be loaded after the helpers it uses. Keeping these categories distinct helps debugging stay focused: first establish that the package loads, then check resource availability and finally verify the feature behavior.

The order of checks matters. When a browser reports that the manifest cannot be loaded, start with manifest syntax and paths before investigating unrelated application logic. Once

the manifest is accepted, inspect any remaining extension errors and test the affected feature on a page where it should activate. This staged approach reduces the chance of changing correct code in response to a packaging problem. It also makes the outcome easier to document because each check has a clear pass or fail result.

For future maintenance, the project should have one canonical location for each icon size and stylesheet. If an asset is moved, the manifest should be updated in the same change. A review of the manifest and project tree before committing can catch accidental renames, deleted files and directory changes. The version should also be checked against the intended release process; in this incident, the version was changed from the prior listed 1.1 to 1.2, but versioning alone does not validate package contents.

The main lesson is that extension reliability depends on agreement between configuration and files on disk. The corrections were small, but they removed blockers at the boundary between the project and the browser. A lightweight asset audit, followed by a real load test and a targeted feature check, provides a practical balance between low-effort prevention and meaningful verification.

A. Failure Prevention and Debugging Order

VI. PREVENTIVE PACKAGING CHECKLIST

Before testing or packaging, check the following:

- Enumerate every file referenced by the manifest, including icons, scripts, stylesheets and other declared resources.
- Confirm that every referenced path exists relative to the extension root and matches the actual directory structure.
- Check that filenames and extensions are consistent and avoid maintaining duplicate assets with ambiguous locations.
- Parse manifest.json as JSON before attempting to load the extension.
- Confirm that content-script dependencies are ordered so required helpers are available before dependent scripts execute.
- Verify that stylesheets contain the selectors used by the extension code and that the rules implement the expected visual behavior.
- Load the unpacked extension in Chrome Developer mode and inspect any error shown by the browser.
- Perform a small functional check of the affected feature, then record which environment and behaviors were actually tested.

Where practical, the path audit can be automated with a small script that reads manifest.json and checks whether each local path exists. Automated validation should complement, not replace, loading the extension in Chrome: a file can exist and still contain an incorrect selector or runtime behavior.

VII. CONCLUSION

The SpoilFree packaging failure resulted from two file-level inconsistencies: icon entries pointed to the wrong location and the declared blur stylesheet was absent. The recorded fix updated the icon paths, recreated styles.css, raised the manifest version to 1.2 and reordered the content scripts. The manifest was parsed as valid JSON and the developer reported a successful load in Developer mode. A pre-load audit of every manifest-referenced path would likely have identified the central problems earlier. Further clean-profile and feature-level checks remain appropriate before treating the package as broadly verified.

REFERENCES

[1] Google, “Manifest file format,” Chrome for Developers. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/reference/manifest

[2] Google, “Content scripts,” Chrome for Developers. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts

[3] Mozilla, “filter,” MDN Web Docs. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/filter